#1 Method `optics` takes a list of points, the (maximum) radius of the dense area defining core points, and the minimum number of points needed in the dense area for a point to be a core point. This function returns a tuple: an array for the order in which points where processed (an array of indices), and a second array with the reachability distance for each point.
#2 Initialize the reachability distance of each point to `null`.
#3 Initialize the ordering for the points: initially an empty array, we'll add the elements as they are processed.
#4 Create a k-d tree to speed range queries. It initializes it with the full dataset.
#5 Cycle through each point in the dataset.
#6 Check if point `p` hasn't yet been processed; if it was processed, just skip it.
#7 Otherwise, add `p` as the next point in the ordering.
#8 Perform the range query collecting all points in the hyper-sphere with center `p`. In this implementation we assume the function does include the point `p` itself (as per the implementation in listing 9.11).
#9 Check if `p` is a core point (hence, if the query at line #8 returns at least `minPoints` points).
#10 Create a new priority queue for the points reachable from `p`, which will be processed next.
#11 Update the queue (by adding all points directly reachable from `p`); also update the reachability distance for `p`'s neighbors.
#12 Cycle through all points in the queue, ordered by their reachability distance.
#13 Extracts from the queue the point `q` with the smallest reachability distance (from the set of points already